

Linguagem C

Revisão

(versão rascunho)

1

Ciclos de repetição

2

❑ Instruções de Repetição

▪ A instrução *while*

```
while (expressão)
    instrução;
```

▪ A instrução *for*

```
for(inicialização; condição; pós-instrução)
    instrução;
```

▪ A instrução *do...while*

```
do
    instrução;
while (expressão)
```

3

❑ Instruções de Repetição

▪ A instrução *while*

○ Resolução

Escreva um programa que mostre no ecrã os 10 primeiros números inteiros.

The diagram illustrates the execution of a C program that prints the first 10 integers using a `while` loop. The code is shown on the left, and the execution flow is detailed on the right.

Code Snippet:

```
#include <stdio.h>

int main() {
    int i;
    i = 1;
    while (i <= 10) {
        printf("\t%d\n", i);
        i = i + 1;
    }
    return 0;
}
```

Execution Flow:

- Initialização:** `i = 1`
- 1º Teste:** `1 <= 10` (Verdade) → Imprime `i = 1` → `i = i + 1`
- 2º Teste:** `2 <= 10` (Verdade) → Imprime `i = 2` → `i = i + 1`
- ...
- 10º Teste:** `10 <= 10` (Verdade) → Imprime `i = 10` → `i = i + 1`
- 11º Teste:** `11 <= 10` (Falso) → Termina o programa

Annotations:

- A - Inicialização:** Points to `i = 1;`
- B - Teste à condição:** Points to `while (i <= 10)`
- Incrementa o valor de i e regressa a B:** Points to `i = i + 1;`
- Os testes vão ocorrer 11 vezes.** (Note: The diagram shows 10 successful tests and 1 failed test, totaling 11 test evaluations.)
- As repetições (i.e., iterações) vão ocorrer 10 vezes**
- A inicialização ocorre apenas uma única vez.**

4

❑ Instruções de Repetição

▪ A instrução *for*

○ Exemplo

Escreva um programa que mostre no ecrã os 10 primeiros números inteiros.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

A - Inicialização.

```
int i;
```

B - Teste à condição.

```
for (i = 1; i <= 10; i++)
```

```
{
```

```
    printf("\t%d\n", i);
```

```
}
```

```
return 0;
```

```
}
```

C - Incrementa o valor de i e regressa a B

A inicialização ocorre apenas uma única vez.

Inicialização: i = 1

1º Teste: 1 <= 10 (Verdade)

Imprime i = 1

i = i + 1

2º Teste: 2 <= 10 (Verdade)

Imprime i = 2

i = i + 1

.

.

10º Teste: 10 <= 10 (Verdade)

Imprime i = 10

i = i + 1

11º Teste: 11 <= 10 (Falso)

Termina o programa

Os testes vão ocorrer 11 vezes.

As repetições vão ocorrer 10 vezes

5

❑ Instruções de Repetição

▪ Os ciclos *for* e *while*

- Observando o código do slide anterior, e escrito utilizando as duas estruturas de repetição:

```
#include <stdio.h>
```

```
int main() {
```

```
int i;
```

```
for (i = 1; i <= 10; i++)
```

```
{
```

```
    printf("\t%d\n", i);
```

```
}
```

```
return 0;
```

```
}
```

O ciclo for separa e identifica todas as componentes

Inicialização.

Teste à condição

Incremento

Ambas as estruturas de repetição, efetuam o teste antes da execução das instruções do corpo da estrutura.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int i;
```

```
i = 1;
```

```
while(i <= 10)
```

```
{
```

```
    printf("\t%d\n", i);
```

```
}
```

```
return 0;
```

```
}
```

Inicialização.

Teste à condição

Incremento

6

❑ Instruções de Repetição

▪ A instrução *do...while*

- A instrução *do...while*, também conhecida como ciclo *do...while*, difere dos ciclos anteriores porque o teste da condição é realizado no fim do corpo do ciclo e não antes.
- Tal como os ciclos anteriores executa uma instrução ou bloco de instruções enquanto uma determinada expressão for verdadeira.
- A sua sintaxe é:

```
do
    instrução;
while (expressão)
```

Expressão a ser avaliada

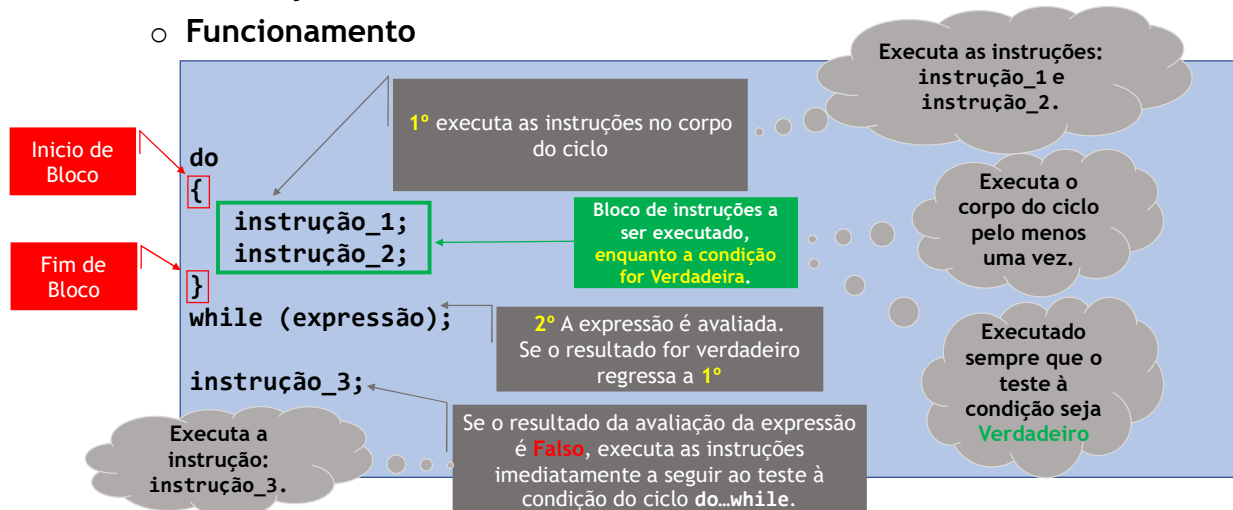
Enquanto esta expressão for **Verdadeira** repete a **instrução**.

7

❑ Instruções de Repetição

▪ A instrução *do...while*

- Funcionamento



8

❑ Instruções de Repetição

▪ A instrução *do...while*

○ Exemplo

Escrever um programa que apresente um menu com opções para realizar cálculos básicos:

```
#include <stdio.h>

int main() {
    int opcao = 0;

    do {
        printf("\n\t+++ CALCULOS BASICOS +++\n");
        printf("\t1. Somar\n");
        printf("\t2. Subtrair\n");
        printf("\t3. Multiplicar\n");
        printf("\t4. Dividir\n");
        printf("\t0. Sair\n");

        printf("\t<Digite opcao>:");
        scanf("%d", &opcao);
    } while (opcao != 0);

    return 0;
}
```

Mostra o Menu no ecrã

```
+++ CALCULOS BASICOS +++
1. Somar
2. Subtrair
3. Multiplicar
4. Dividir
0. Sair
<Digite opcao>:
```

Recolhe a opcao

Enquanto a opcao for diferente de zero, repete as instruções do corpo do ciclo

Caso a opcao seja igual a zero, termina o ciclo

9

Arrays (vetores)

10

❑ Vetores

▪ Declaração de vetores

- A sintaxe para declarar um vetor com uma única dimensão é:

tipo identificador[nº de elementos]

Corresponde ao tipo de dados de cada um dos elementos do vetor

Indica o nome pelo qual este vetor vai ser conhecido

Valor constante que indica quantos elementos tem o vetor

11

❑ Vetores

▪ Declaração de vetores e atribuição de valores

- Declarar um vetor com 6 números inteiros

```
int numeros[6];
```

- Atribuir seis valores inteiros ao vetor previamente criado

```
numeros[0] = 16;
numeros[1] = 2;
numeros[2] = 32;
numeros[3] = 4;
numeros[4] = 64;
numeros[5] = 128;
```

Atribuição dos 6 valores inteiros a cada uma das posições

12

❑ Vetores

▪ Utilização de vetores

○ Exemplo

Escreva um programa em Linguagem C, que realize a recolha da temperatura registada de hora a hora, durante 12 horas. De seguida deverá de mostrar os valores da temperatura registada em cada hora, e no final deverá de apresentar a temperatura média.

The screenshot shows a C program for collecting temperature data over 12 hours. The code is as follows:

```
#include <stdio.h>

int main() {
    float temperatura[12];
    float soma_temperatura = 0.0F;
    int i;
    // Recolha da temperatura registada a cada hora
    for (i = 0; i < 12; i++){
        printf("Introduza a temperatura da hora %02d:", (i + 1));
        scanf("%f", &temperatura[i]);
    }
    .
    [continua ...]
}
```

Annotations and a terminal output are included:

- Número de elementos**: Points to the array size `12` in `temperatura[12]`.
- Repete a recolha da temperatura 12 vezes.**: Points to the `for` loop.
- Guarda o valor recolhido no índice `i` do vetor**: Points to `&temperatura[i]` in the `scanf` call.
- Índice `i`, que vai variar entre 0 e 11, o que perfaz 12 posições**: Points to the loop condition `i < 12`.
- Não faz sentido perguntar ao utilizador para introduzir a temperatura da hora 0.**: A thought bubble pointing to the loop starting at `i = 0`.
- Terminal output**: Shows a sequence of prompts "Introduza a temperatura da hora 01:22.3" through "12:26.2" and a final message "Process returned 0 (0x0) execution time : 46.193 s Press any key to continue."

13

❑ Vetores

▪ Definição de constantes na utilização de vetores

○ Exemplo

E se, devido a alteração nos requisitos, fosse necessário recolher a temperatura de cada hora, apenas durante 8 horas ?

The screenshot shows a C program using a constant `MAX_HORAS` for the array size. The code is as follows:

```
#include <stdio.h>

#define MAX_HORAS 8

int main() {
    float temperatura[MAX_HORAS];
    float soma_temperatura = 0.0F;
    int i;
    // Recolha da temperatura registada a cada hora
    for (i = 0; i < MAX_HORAS; i++){
        printf("Introduza a temperatura da hora %02d:", (i + 1));
        scanf("%f", &temperatura[i]);
    }
    .
    [continua ...]
}
```

Annotations and a thought bubble are included:

- Define-se um símbolo `MAX_HORAS` com um valor constante igual a 8**: Points to the `#define MAX_HORAS 8` line.
- O máximo de elementos depende agora do valor de `MAX_HORAS`**: Points to the array declaration `temperatura[MAX_HORAS]`.
- Também o número de iterações, para recolher os valores depende do valor do símbolo `MAX_HORAS`**: Points to the loop condition `i < MAX_HORAS`.
- Assim, se fosse necessário alterar para outro valor, bastava apenas alterar o valor deste símbolo.**: A thought bubble.

14

❑ Vetores

▪ Matrizes e vetores Multidimensionais

- A declaração de vetores unidimensionais é um caso particular da declaração de vetores com qualquer número de dimensões.
- A declaração de um qualquer vetor com n dimensões é realizada através da seguinte sintaxe:

```
tipo identificador[dim1] [dim2] [...] [dimn]
```

15

Tipos Compostos (estruturas)

16

Struct

- Para definir uma estrutura e declarar uma **variável**:

```
struct [nomeDaEstrutura]
{
    tipo1 campo1, campo2;
    ...
    tipon campon;
} nomeDaVariavel;
```

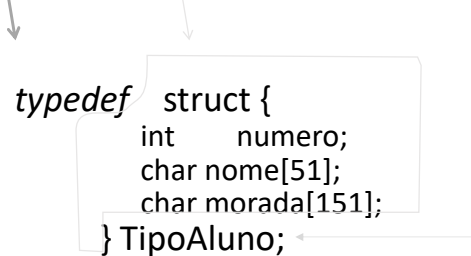
17

17

Definição de tipos de dados em C

- Para definir novos tipos utiliza-se a instrução *typedef*

typedef definição_do_tipo NomeDoTipo;



```
typedef struct {
    int    numero;
    char nome[51];
    char morada[151];
} TipoAluno;
```

- Nota: utiliza-se muito com structs para definir novos tipos compostos, mas a instrução *typedef* pode ser usada noutros casos com outras estruturas de dados: *typedef enum{true, false} bool;*

18

18

Exemplo aluno com definição de tipo

```
#include <stdio.h>
#include <string.h>

// fora do main para estar disponível para todos os subprogramas
typedef struct {
    int      numero;
    char     nome[51];
    char     morada[151];
} TipoAluno; // nome do tipo

int main(void){
    TipoAluno a; // declaração de uma variável do tipo TipoAluno
    TipoAluno b = {2,"Manel","outra rua"}; // declaração e iniciação de outra var c/ iniciação

    a.numero = 1;
    strcpy(a.nome,"Ana Maria");
    strcpy(a.morada,"Rua .... Portugal");

    printf("Numero = %d\nNome = %s\nMorada = %s\n\n",a.numero,a.nome,a.morada);
    printf("Numero = %d\nNome = %s\nMorada = %s\n",b.numero,b.nome,b.morada);
    return 0;
}
```

19

19

Exemplo aluno com definição de tipo

- **TipoAluno a;**
- **int i;**
- **float valor;**
- ...
- **Declara-se como qualquer outro tipo.**

20

20

Exercício 1

- Defina um tipo denominado **TipoData** que permita representar datas com **três campos inteiros**:
 - ano, dia e mês.
- Escreva um pequeno programa em C que:
 - peça ao utilizador que insira uma data,
 - guarde os valores lidos para uma variável do tipo **TipoData**
 - e imprima a data no seguinte formato: dia/mês/ano.

21

21

Solução para exercício 1

```
#include <stdio.h>
#include <string.h>

typedef struct {
    int ano,mes,dia;
} TipoData;

int main(void){

    TipoData data;

    printf("Ano = ");
    scanf("%d", &data.ano);
    printf("Mes = ");
    scanf("%d", &data.mes);
    printf("Dia = ");
    scanf("%d", &data.dia);
    printf("%d/%d/%d\n", data.dia, data.mes, data.ano);

    return 0;
}
```

22

22

Apontadores

23

O que é um apontador? R: guarda um endereço de memória

Como se **declara** um apontador?

R: é necessário informar o compilador , **colocando um ***

Sintaxe:

```
tipodeVariável *nomeDaVariável;
```



```
int num;  
int *aptNum;
```

Exemplos:

```
int *apontador1, naoApontador;  
int *apontador2, *apontador3;
```

Gonçalo Piedade

24

24

O que é um apontador? R: guarda um endereço de memória

Operadores: & e *

& (*endereço de*)

***** (*conteúdo de* ou *apontado por*)

```
int num;
int *aptNum;

num = 3;
aptNum = &num;
```

&num: "endereço de num"

Como é guardado na memória?*

(endereço) &	Nome	Valor *
229		
230	num	3
234	aptNum	230
238		

*NOTA: Para o exemplo, assumo-se que um apontador ocupa 4 bytes na memória (4 setores)